

ASA__Week_5_Network_Analytics

April 29, 2026

0.1 Analytics Specializations & Applications - Week 5

0.2 # Network Analytics - Example with Python and NetworkX

Dr Georgiana Nica-Avram - University of Nottingham [mail](#) [web](#)

1 Downloading our edge list

```
[ ]: !wget -O BUSI4370_week5_individual_task_edgelist.xlsx "https://drive.google.com/uc?export=download&id=1lPODGGMqFm524q0CyDIjONA8Mxs5PS92"

# You can then check that the file is here by listing the content of the
current directory
!ls
```

--2026-03-27 10:12:10--

https://drive.google.com/uc?export=download&id=1lPODGGMqFm524q0CyDIjONA8Mxs5PS92

Resolving drive.google.com (drive.google.com)... 192.178.223.139,

192.178.223.138, 192.178.223.113, ...

Connecting to drive.google.com (drive.google.com)|192.178.223.139|:443...

connected.

HTTP request sent, awaiting response... 303 See Other

Location: https://drive.usercontent.google.com/download?id=1lPODGGMqFm524q0CyDIjONA8Mxs5PS92&export=download [following]

--2026-03-27 10:12:10-- https://drive.usercontent.google.com/download?id=1lPODGGMqFm524q0CyDIjONA8Mxs5PS92&export=download

Resolving drive.usercontent.google.com (drive.usercontent.google.com)...

142.251.29.132, 2a00:1450:4009:c15::84

Connecting to drive.usercontent.google.com

(drive.usercontent.google.com)|142.251.29.132|:443... connected.

HTTP request sent, awaiting response... 200 OK

Length: 20733 (20K) [application/octet-stream]

Saving to: 'BUSI4370_week5_individual_task_edgelist.xlsx'

BUSI4370_week5_indi 100%[=====>] 20.25K --.-KB/s in 0.01s

2026-03-27 10:12:12 (1.40 MB/s) - 'BUSI4370_week5_individual_task_edgelist.xlsx'
saved [20733/20733]

ASA_Week_08_Case_Study_Exercises.ipynb
 ASA_Week_5_Network_Analytics.ipynb
 ASA_practical_04_Basket_analysis_with_topic_modelling.ipynb
 BUSI4370_week5_individual_task_edgelist.xlsx
 IDF.png
 TF.png
 Week_2_Case_Study_Exercises.ipynb
 Week_3_Case_Study_Exercises.ipynb
 Week_7_Text_Analytics_case_study_with_solutions.ipynb
 boosting.png
 coursework-1
 credit_card_history.csv
 html_cleaner.py
 movie_data.tsv
 nlab_logo.png
 renders.py
 vsm.png
 week2_data.zip
 week3_data.zip
 wholesale_customer_data.csv

```
[ ]: import pandas as pd

# Load in the data
data = pd.ExcelFile("BUSI4370_week5_individual_task_edgelist.xlsx")
print(data.sheet_names)
data_df = data.parse("Clean_edgelist")

# Let's have a look at the file
data_df = data_df.dropna().reset_index(drop=True)
print("Number of datapoints: ", data_df.shape[0]) # this corresponds to the
↳ number of edges or messages in the file

# The table should have two columns, from and to
data_df.head()
```

```
['Individual task edgelist', 'Clean_edgelist']
Number of datapoints: 107
```

```
[ ]:      from      to
0  Paras Yadav  Shivangi M.
1  Paras Yadav    DEPIN S
2  Paras Yadav    Paras LG
3  Paras Yadav  Konstantina A.
4  Paras Yadav    Noor B.
```

```
[ ]: data_df.isnull().sum()
```

```
[ ]: from    0
      to      0
      dtype: int64
```

```
[ ]: data_df.dropna()
```

```
[ ]:
      from      to
0   Paras Yadav  Shivangi M.
1   Paras Yadav    DEPIN S
2   Paras Yadav   Paras LG
3   Paras Yadav  Konstantina A.
4   Paras Yadav    Noor B.
..      ...      ...
102  Rehan A.    Yuehan Luo
103  Rehan A.    Noor B.
104  Rehan A.  Konstantina A.
105  Rehan A.   Harrison C.
106  Rehan A.   Emmanuel P.
```

```
[107 rows x 2 columns]
```

2 Importing necessary modules

NetworkX is a Python package for the creation, manipulation, and study of complex networks.

```
[ ]: import networkx as nx
      import matplotlib.pyplot as plt
```

3 We will start by loading our data from the CSV file.

The edges in a graph express relationships among its nodes. Sometimes, these relationships are naturally symmetric or there is no orientation in the direction of these relationships (e.g., relationships of collegiality, where you are class-mates with everyone on the MSc course, and so is everyone on the programme to you).

Other times, these relationships are not symmetric, in which case it is desirable to use directed arcs to show the directionality of relationships/ behaviours.

Next, we will load our digital communication data as a directed graph to indicate who has emailed whom among your MSc peers.

```
[ ]: G_emails = nx.from_pandas_edgelist(data_df, source = "from", target="to",
      ↪create_using=nx.DiGraph())
```

#Accessing nodes and edges

The edgelist returns 2 Python lists: a list of nodes (or student name) and a list of edges (edges, or sent emails). In NetworkX, you can put these two lists together into a single network object that understands how nodes and edges are related.

```
[ ]: print('This is a list of edges in our classroom network, indicating who has_
      ↪emailed whom: ', G_emails.edges)
      print('There are %d edges in the graph.' % len(G_emails.edges))
```

This is a list of edges in our classroom network, indicating who has emailed whom: [('Paras Yadav', 'Shivangi M.'), ('Paras Yadav', 'DEPIN S'), ('Paras Yadav', 'Paras LG'), ('Paras Yadav', 'Konstantina A.'), ('Paras Yadav', 'Noor B.'), ('Paras Yadav', 'Emmanuel P.'), ('Paras Yadav', 'Shreyash M.'), ('Paras Yadav', 'Ashutosh S.'), ('Paras Yadav', 'Georgiana N.'), ('Shivangi M.', 'Ashutosh S.'), ('Shivangi M.', 'Shreyash M.'), ('Shivangi M.', 'Paras LG'), ('Shivangi M.', 'Paras Yadav'), ('Shivangi M.', 'Konstantina A.'), ('Shivangi M.', 'DEPIN S'), ('Shivangi M.', 'JIayi Wu\n'), ('Shivangi M.', 'Parima P.'), ('Shivangi M.', 'Georgiana N.'), ('DEPIN S', 'Georgiana N.'), ('DEPIN S', 'Efe U.'), ('DEPIN S', 'Paras LG'), ('DEPIN S', 'Paras Yadav'), ('DEPIN S', 'Ling'), ('DEPIN S', 'Lucas'), ('DEPIN S', 'Ashutosh S.'), ('DEPIN S', 'Shreyash M.'), ('DEPIN S', 'Noor B.'), ('DEPIN S', 'Emmanuel P.'), ('Paras LG', 'DEPIN S'), ('Paras LG', 'Efe U.'), ('Paras LG', 'Shivangi M.'), ('Paras LG', 'Parima P.'), ('Paras LG', 'Georgiana N.'), ('Paras LG', 'Paras Yadav'), ('Konstantina A.', 'Georgiana N.'), ('Konstantina A.', 'Shivangi M.'), ('Konstantina A.', 'Efe U.'), ('Konstantina A.', 'Yuehan Luo'), ('Konstantina A.', 'Noor B.'), ('Konstantina A.', 'Xinyi Chen'), ('Noor B.', 'Konstantina A.'), ('Noor B.', 'Efe U.'), ('Noor B.', 'Paras Yadav'), ('Noor B.', 'Ashutosh S.'), ('Noor B.', 'Shreyash M.'), ('Noor B.', 'Georgiana N.'), ('Emmanuel P.', 'Yuehan Luo'), ('Emmanuel P.', 'nick'), ('Emmanuel P.', 'Paras Yadav'), ('Shreyash M.', 'Georgiana N.'), ('Shreyash M.', 'Efe U.'), ('Shreyash M.', 'Paras Yadav'), ('Shreyash M.', 'DEPIN S'), ('Shreyash M.', 'Shivangi M.'), ('Shreyash M.', 'Ashutosh S.'), ('Shreyash M.', 'Lucas'), ('Ashutosh S.', 'Paras Yadav'), ('Ashutosh S.', 'Shreyash M.'), ('Ashutosh S.', 'Shivangi M.'), ('Ashutosh S.', 'Lucas'), ('Ashutosh S.', 'DEPIN S'), ('Efe U.', 'Emmanuel P.'), ('Efe U.', 'DEPIN S'), ('Efe U.', 'Paras LG'), ('Efe U.', 'Shreyash M.'), ('Efe U.', 'Noor B.'), ('Efe U.', 'Konstantina A.'), ('Efe U.', 'Yuehan Luo'), ('Efe U.', 'Georgiana N.'), ('Parima P.', 'Konstantina A.'), ('Parima P.', 'Noor B.'), ('Parima P.', 'JIayi Wu\n'), ('Yuehan Luo', 'nick'), ('Yuehan Luo', 'Xinyi Chen'), ('Yuehan Luo', 'Efe U.'), ('Yuehan Luo', 'JIayi Wu\n'), ('Yuehan Luo', 'Georgiana N.'), ('Yuehan Luo', 'Konstantina A.'), ('Ling', 'Lucas'), ('Ling', 'nick'), ('Ling', 'DEPIN S'), ('Lucas', 'nick'), ('Lucas', 'Ashutosh S.'), ('Lucas', 'Ling'), ('Lucas', 'DEPIN S'), ('JIayi Wu\n', 'Shivangi M.'), ('JIayi Wu\n', 'Parima P.'), ('JIayi Wu\n', 'Yuehan Luo'), ('JIayi Wu\n', 'Xinyi Chen'), ('nick', 'Ling'), ('nick', 'Xinyi Chen'), ('nick', 'Yuehan Luo'), ('nick', 'Lucas'), ('Xinyi Chen', 'Yuehan Luo'), ('Xinyi Chen', 'JIayi Wu\n'), ('Xinyi Chen', 'nick'), ('Rehan A.', 'Paras Yadav'), ('Rehan A.', 'Efe U.'), ('Rehan A.', 'Paras LG'), ('Rehan A.', 'Georgiana N.'), ('Rehan A.', 'DEPIN S'), ('Rehan A.', 'Yuehan Luo'), ('Rehan A.', 'Noor B.'), ('Rehan A.', 'Konstantina A.'), ('Rehan A.', 'Harrison C.'), ('Rehan A.', 'Emmanuel P.')]

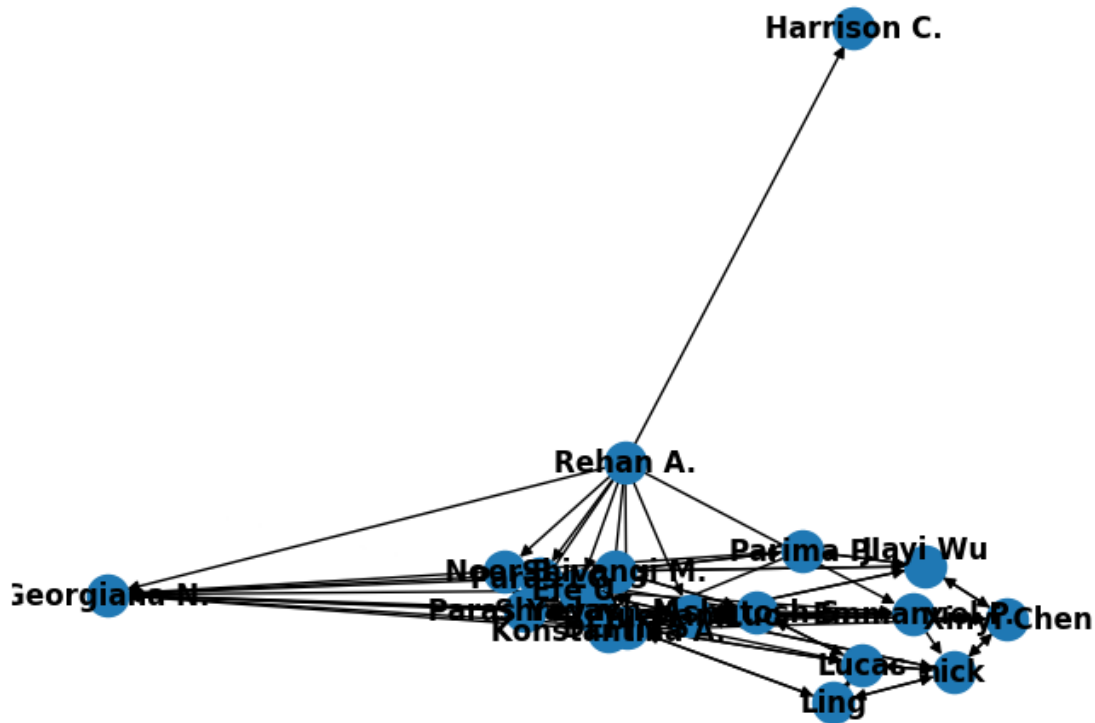
There are 106 edges in the graph.

4 Plotting our network

After seeing what the dataset looks like, it's important to see what the network looks like. You can generate a representation of your network in many different ways as you will see here: <https://networkx.org/documentation/stable/reference/drawing.html>

Below are two examples.

```
[ ]: #-- There are many ways to visualize a network, and a force-directed layout, of  
      ↳ which the below figure is an example, is among the most common.  
  
      nx.draw(G_emails, nx.spring_layout(G_emails, scale=2), with_labels=True,  
            ↳ font_weight='bold') # scale=2 doubles the distance between all nodes
```

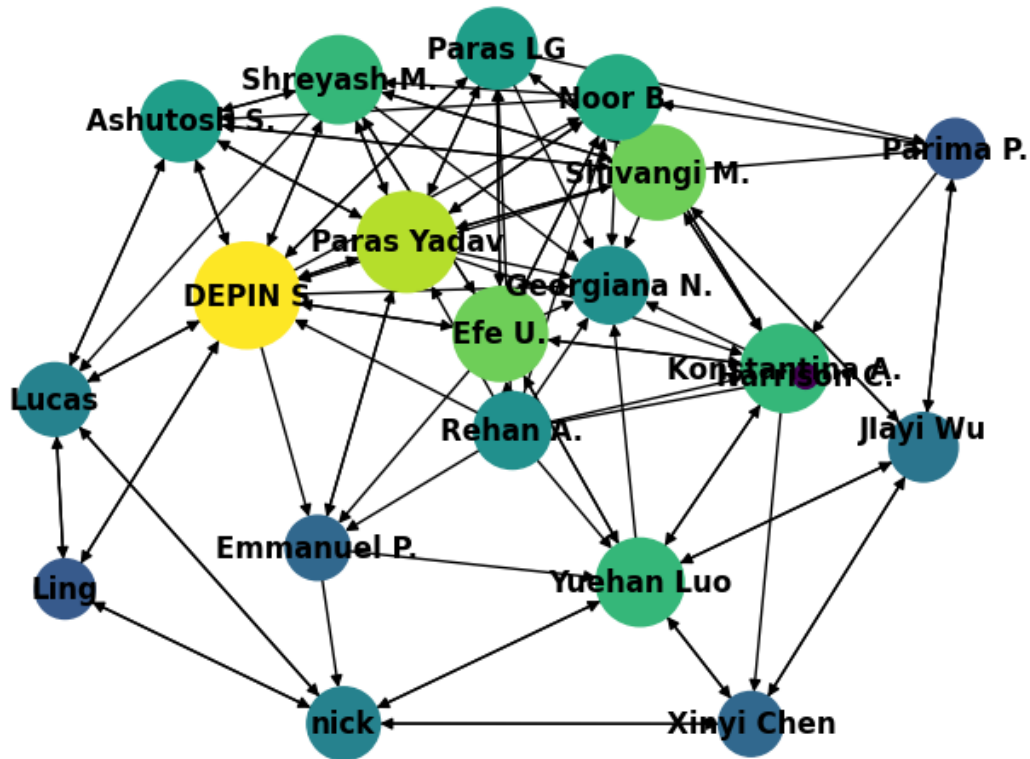


```
[ ]: ##-- Kamada-Kawai force-directed layout.  
import numpy as np  
deg Centrality = nx.degree_Centrality(G_emails)  
centrality = np.fromiter(deg_Centrality.values(), float) # create a new  
one-dimensional array from the deg_Centrality object
```

```

nx.draw_kamada_kawai(G_emails, with_labels=True, font_weight='bold', node_size=
↪centrality*2e3, node_color=centrality) # node size and colour denote its
↪degree centrality

```



5 Computing characteristics of our network

Below are a few examples to obtain characteristics of the network. A full list of available measures can be seen here: <https://networkx.org/documentation/stable/reference/algorithms/index.html>

5.1 Density

A good metric to begin with is the network density, or the ratio of actual edges to all possible edges in the network. In an directed network like this one, there could be an edge (email) between any two nodes (students), but as you saw in the visualization, only a few of those possible edges are actually present. Network density gives you a quick sense of how closely knit your network is.

```

[ ]: #-- Density

nx.density(G_emails)

```

```
!-- A value of 0 would mean that there are no connections at all, and a value  
of 1 would indicate that all possible edges are present, or everyone has  
emailed everyone else (a perfectly connected network).
```

```
[ ]: 0.2789473684210526
```

5.2 Triadic closure

Another metric you will calculate on this network concerns the concept of triadic closure. Triadic closure supposes that if two people know the same person, they are likely to know each other. The number of these enclosed triangles in the network can be used to find clusters and communities of individuals that all know each other fairly well.

One way of measuring triadic closure is called clustering coefficient because of this clustering tendency.

Another is transitivity, or the ratio of all triangles over all possible triangles. A possible triangle exists when one person knows two people. So transitivity, like density, expresses how interconnected a graph is in terms of a ratio of actual over possible connections.

```
[ ]: print("This network's average clustering coefficient is %f." % nx.  
      average_clustering(G_emails))  
  
print('Its transitivity is %f. ' % nx.transitivity(G_emails))
```

This network's average clustering coefficient is 0.403740.
Its transitivity is 0.395498.

5.3 Reciprocity

```
[ ]: !-- Algorithms to calculate reciprocity in a directed graph (in this case,  
      two-way digital communication).  
  
nx.reciprocity(G_emails)
```

```
[ ]: 0.6792452830188679
```

6 Which nodes are most important

After getting some basic measures of the entire network, a good next step is to find which nodes are the most important ones in your network. In network analysis, measures of the importance of nodes are referred to as centrality measures.

6.1 Measures of degree

Degree is the simplest and the most common way of finding important nodes. A node's degree is the sum of its edges. The nodes with the highest degree in a social network are the people who know the most people. These nodes are often referred to as *hubs*.

```
[ ]: ## Degree, or how many conversations each person has had
G_emails.degree()
```

```
[ ]: DiDegreeView({'Paras Yadav': 17, 'Shivangi M.': 15, 'DEPIN S': 19, 'Paras LG': 11, 'Konstantina A.': 13, 'Noor B.': 12, 'Emmanuel P.': 7, 'Shreyash M.': 13, 'Ashutosh S.': 11, 'Efe U.': 15, 'Parima P.': 6, 'Georgiana N.': 10, 'Yuehan Luo': 13, 'Ling': 6, 'Lucas': 9, 'JIayi Wu\n': 8, 'nick': 9, 'Xinyi Chen': 7, 'Rehan A.': 10, 'Harrison C.': 1})
```

```
[ ]: # Who are the top 3 communicators?
sorted(G_emails.degree, key=lambda x: x[1], reverse=True)[:3]
```

```
[ ]: [('DEPIN S', 19), ('Paras Yadav', 17), ('Shivangi M.', 15)]
```

```
[ ]: ## In-degree, or how many emails has each person received
G_emails.in_degree()
```

```
[ ]: InDegreeView({'Paras Yadav': 8, 'Shivangi M.': 6, 'DEPIN S': 9, 'Paras LG': 5, 'Konstantina A.': 7, 'Noor B.': 6, 'Emmanuel P.': 4, 'Shreyash M.': 6, 'Ashutosh S.': 6, 'Efe U.': 7, 'Parima P.': 3, 'Georgiana N.': 10, 'Yuehan Luo': 7, 'Ling': 3, 'Lucas': 5, 'JIayi Wu\n': 4, 'nick': 5, 'Xinyi Chen': 4, 'Rehan A.': 0, 'Harrison C.': 1})
```

```
[ ]: ## Out-degree, or how many emails has each person sent
G_emails.out_degree()
```

```
[ ]: OutDegreeView({'Paras Yadav': 9, 'Shivangi M.': 9, 'DEPIN S': 10, 'Paras LG': 6, 'Konstantina A.': 6, 'Noor B.': 6, 'Emmanuel P.': 3, 'Shreyash M.': 7, 'Ashutosh S.': 5, 'Efe U.': 8, 'Parima P.': 3, 'Georgiana N.': 0, 'Yuehan Luo': 6, 'Ling': 3, 'Lucas': 4, 'JIayi Wu\n': 4, 'nick': 4, 'Xinyi Chen': 3, 'Rehan A.': 10, 'Harrison C.': 0})
```

```
[ ]: avg_in_degree = 0
for x in G_emails.in_degree():
    avg_in_degree += x[1]

avg_in_degree = avg_in_degree / len(G_emails.in_degree())
print('On average, your class-mates have sent %d email(s).' % avg_in_degree)
```

On average, your class-mates have sent 5 email(s).

```
[ ]: ## Some communicate more frequently than average.
for x in G_emails.in_degree():
    if x[1] > avg_in_degree:
        print(x)
```

```
('Paras Yadav', 8)
('Shivangi M.', 6)
('DEPIN S', 9)
```



```
( 'Konstantina A.', 7)
( 'Noor B.', 6)
( 'Shreyash M.', 6)
( 'Ashutosh S.', 6)
( 'Efe U.', 7)
( 'Georgiana N.', 10)
( 'Yuehan Luo', 7)
```

6.2 Closeness centrality

Closeness centrality is calculated as the reciprocal of the sum of the length of the shortest paths between the node and all other nodes in the graph. Thus, the more central a node is, the closer it is to all other nodes.

```
[ ]: ## Closeness centrality
      nx.closeness centrality(G_emails)

[ ]: { 'Paras Yadav': 0.5245009074410163,
      'Shivangi M.': 0.5070175438596491,
      'DEPIN S': 0.5633528265107213,
      'Paras LG': 0.4753289473684211,
      'Konstantina A.': 0.5245009074410163,
      'Noor B.': 0.5070175438596491,
      'Emmanuel P.': 0.4473684210526316,
      'Shreyash M.': 0.5070175438596491,
      'Ashutosh S.': 0.5070175438596491,
      'Efe U.': 0.5633528265107213,
      'Parima P.': 0.42251461988304095,
      'Georgiana N.': 0.6558704453441295,
      'Yuehan Luo': 0.5432330827067668,
      'Ling': 0.42251461988304095,
      'Lucas': 0.4753289473684211,
      'Jlly Wu\n': 0.4473684210526316,
      'nick': 0.4609250398724083,
      'Xinyi Chen': 0.4473684210526316,
      'Rehan A.': 0.0,
      'Harrison C.': 0.05263157894736842}
```

6.3 Betweenness centrality

Betweenness centrality, which is also expressed on a scale of 0 to 1, finds nodes that connect two otherwise disparate parts of a network. If you're the only thing connecting two clusters, every communication between those clusters has to pass through you. In contrast to a *hub* (nodes with the highest degree), this sort of node is often referred to as a *broker*.

```
[ ]: ## Betweenness centrality
      nx.betweenness centrality(G_emails)
```

```
[ ]: {'Paras Yadav': 0.05896552278131224,
      'Shivangi M.': 0.08427050532313689,
      'DEPIN S': 0.11640898899670829,
      'Paras LG': 0.022935811751601223,
      'Konstantina A.': 0.048508289955658375,
      'Noor B.': 0.03328236902798306,
      'Emmanuel P.': 0.015458785853522693,
      'Shreyash M.': 0.021739177002334898,
      'Ashutosh S.': 0.025041414076501796,
      'Efe U.': 0.06985135559696963,
      'Parima P.': 0.009781397939292676,
      'Georgiana N.': 0.0,
      'Yuehan Luo': 0.08113535262658068,
      'Ling': 0.008698830409356723,
      'Lucas': 0.031186406186406185,
      'Jlayi Wu\n': 0.03884658226763489,
      'nick': 0.048945726138708595,
      'Xinyi Chen': 0.01593763611307471,
      'Rehan A.': 0.0,
      'Harrison C.': 0.0}
```

```
[ ]: ## Potential brokers
brokerage_dict = dict(nx.betweenness centrality(G_emails))

ordered_brokers = sorted(brokerage_dict, key=brokerage_dict.get, reverse=True)
print(ordered_brokers[:3])
```

```
['DEPIN S', 'Shivangi M.', 'Yuehan Luo']
```

```
[ ]: ## Shortest path computes the shortest paths and path lengths between nodes in
      the graph.
      ## What's the shortest path you could reach someone you haven't talked to
      before?

nx.shortest_path(G_emails, source = 'Paras Yadav', target = 'Shreyash M.') #
      replace 'a' and 'b' with actual node labels/ student names
```

```
[ ]: ['Paras Yadav', 'Shreyash M.']
```

7 Time for some experimentation on a different network

7.1 Zachary karate club

This is the well-known and much-used Zachary karate club network. The data was collected from the members of a university karate club by Wayne Zachary in 1977. Each node represents a member of the club, and each edge represents a tie between two members of the club. The network is undirected. An often discussed problem using this dataset is to find the two groups of people into which the karate club split after an argument between two teachers.

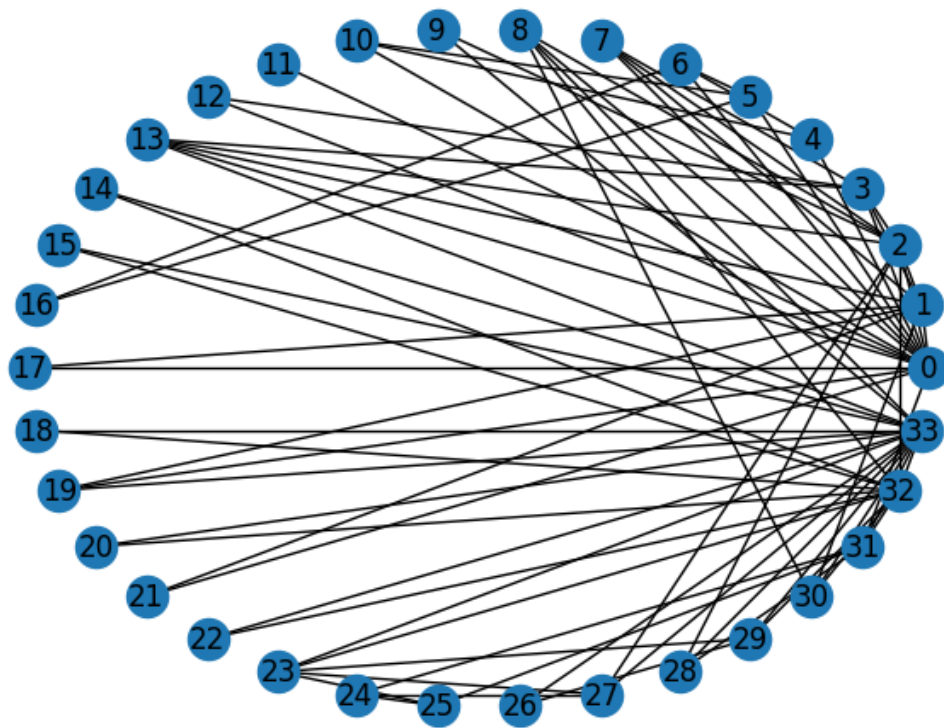
```
[ ]: G_karate = nx.karate_club_graph() # Load the graph from networkx's library

print('Node Degree')
for v in G_karate:
    print(f"{v:4} {G_karate.degree(v):6}")

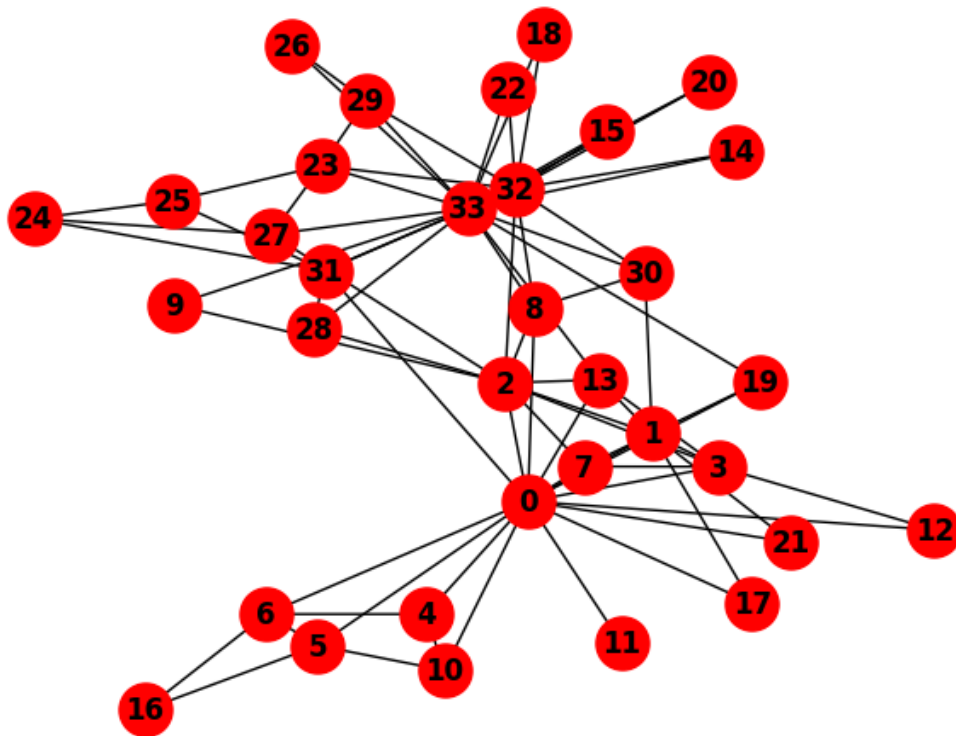
nx.draw_circular(G_karate, with_labels=True)
```

Node Degree

0	16
1	9
2	10
3	6
4	3
5	4
6	4
7	4
8	5
9	2
10	3
11	1
12	2
13	5
14	2
15	2
16	2
17	2
18	2
19	3
20	2
21	2
22	2
23	5
24	3
25	3
26	2
27	4
28	3
29	4
30	4
31	6
32	12
33	17



```
[ ]: ##-- Task 1: Use a force-directed layout to draw Zachary's network. Experiment
      ↪ with different node size and colours to get a sense of network dynamics.
      nx.draw(G_karate, nx.spring_layout(G_karate, scale=10), with_labels=True,
      ↪ font_weight='bold', node_size=500, node_color='RED')
```



```
[ ]: ##-- Task 2: Start computing global network characteristics

##-- How does density differ from your own class network?
print("Zachary's karate network density: ", nx.density(G_karate) )

##-- Average clustering coefficient
print('Avg. clustering coefficient: ', nx.average_clustering(G_karate) )

##-- Network transitivity
print('Network transitivity: ', nx.transitivity(G_karate) )
```

Zachary's karate network density: 0.13903743315508021
 Avg. clustering coefficient: 0.5706384782076823
 Network transitivity: 0.2556818181818182

```
[ ]: ##-- Which nodes are most important? Please note this network is undirected, ↵
↵and as such has no in out out-going edges
degrees = dict(G_karate.degree())

##-- Degree
print(degrees)
```

```

#-- Average degree
avg_degree = sum(degrees.values()) / len(degrees)
print("Average degree:", avg_degree)
#-- Some have more connections than average.
above_avg_degree_nodes = [node for node, degree in degrees.items() if degree >
    ↪ avg_degree]
print("Nodes with degree greater than average:", above_avg_degree_nodes)

```

```

{0: 16, 1: 9, 2: 10, 3: 6, 4: 3, 5: 4, 6: 4, 7: 4, 8: 5, 9: 2, 10: 3, 11: 1, 12:
2, 13: 5, 14: 2, 15: 2, 16: 2, 17: 2, 18: 2, 19: 3, 20: 2, 21: 2, 22: 2, 23: 5,
24: 3, 25: 3, 26: 2, 27: 4, 28: 3, 29: 4, 30: 4, 31: 6, 32: 12, 33: 17}
Average degree: 4.588235294117647
Nodes with degree greater than average: [0, 1, 2, 3, 8, 13, 23, 31, 32, 33]

```

[]: *##-- Measures of centrality*

```

#-- Closeness centrality. How does the centrality of this network's top 5
    ↪ members differ from your own network?
closeness centrality = nx.closeness centrality(G_karate)
print('Closeness centrality: ', closeness centrality)

#-- Betweenness centrality
betweenness centrality = nx.betweenness centrality(G_karate)
print('Betweenness centrality: ', betweenness centrality)

#-- Potential brokers in this network
brokerage_dict = {node: closeness centrality[node] *
    ↪ betweenness centrality[node] for node in G_karate.nodes()}
ordered_brokers = sorted(brokerage_dict.items(), key=lambda x: x[1],
    ↪ reverse=True)[:3]
print('Top 3 potential brokers:', ordered_brokers)

```

```

Closeness centrality: {0: 0.5689655172413793, 1: 0.4852941176470588, 2:
0.559322033898305, 3: 0.4647887323943662, 4: 0.3793103448275862, 5:
0.38372093023255816, 6: 0.38372093023255816, 7: 0.44, 8: 0.515625, 9:
0.4342105263157895, 10: 0.3793103448275862, 11: 0.36666666666666664, 12:
0.3707865168539326, 13: 0.515625, 14: 0.3707865168539326, 15:
0.3707865168539326, 16: 0.28448275862068967, 17: 0.375, 18: 0.3707865168539326,
19: 0.5, 20: 0.3707865168539326, 21: 0.375, 22: 0.3707865168539326, 23:
0.39285714285714285, 24: 0.375, 25: 0.375, 26: 0.3626373626373626, 27:
0.4583333333333333, 28: 0.4520547945205479, 29: 0.38372093023255816, 30:
0.4583333333333333, 31: 0.5409836065573771, 32: 0.515625, 33: 0.55}
Betweenness centrality: {0: 0.43763528138528146, 1: 0.053936688311688304, 2:
0.14365680615680618, 3: 0.011909271284271283, 4: 0.0006313131313131313, 5:
0.02998737373737374, 6: 0.029987373737373736, 7: 0.0, 8: 0.05592682780182781, 9:
0.0008477633477633478, 10: 0.0006313131313131313, 11: 0.0, 12: 0.0, 13:
0.04586339586339586, 14: 0.0, 15: 0.0, 16: 0.0, 17: 0.0, 18: 0.0, 19:
0.03247504810004811, 20: 0.0, 21: 0.0, 22: 0.0, 23: 0.017613636363636363, 24:

```

0.0022095959595959595, 25: 0.0038404882154882154, 26: 0.0, 27:
0.02233345358345358, 28: 0.0017947330447330447, 29: 0.0029220779220779218, 30:
0.014411976911976909, 31: 0.13827561327561325, 32: 0.145247113997114, 33:
0.30407497594997596}
Top 3 potential brokers: [(0, 0.24899938423645326), (33, 0.16724123677248678),
(2, 0.08035041700295938)]